



FileGuard

File Integrity Monitoring Tool

Week 3 Project

Instructor: Masooma Ali

Student Name:	Zeeshan Haider
Domain:	Vulnerability Assessment
Submission Date:	23 th May 2026

Pakistan Computer Emergency Response Team, PKCERT

FileGuard

File Integrity Monitoring Tool

Cybersecurity Internship Project Report

Programming Language: Python

Project Type: Cybersecurity Tool

Primary Function: File Integrity Monitoring

1.Objective

FileGuard is a lightweight File Integrity Monitoring (FIM) tool developed using Python. Its primary objective is to detect unauthorized or unexpected changes in files by comparing their current SHA-256 cryptographic hashes with a previously created trusted baseline.

The tool scans a user-selected folder and its subfolders. It identifies:

- Unchanged files
- Modified files
- Newly created files
- Deleted files

FileGuard provides a simple and practical way to monitor the integrity of important files and detect changes that may indicate accidental modification, unauthorized activity, or possible malicious activity.

The project focuses on implementing a cybersecurity concept using simple and understandable programming techniques.

2. Problem Statement

Files stored on computers can be modified, added, or deleted without the owner's knowledge. In a large directory containing many files and subfolders, manually checking whether every file has changed is difficult and time-consuming.

For example, if an important configuration file is modified by an unauthorized user or malware, it may be difficult to identify the change manually.

FileGuard solves this problem by creating a trusted baseline of the selected folder. The baseline stores the relative path of each file along with its SHA-256 hash.

During a later scan, the tool calculates the current hash of every file again and compares it with the original hash.

The results are classified as:

- **UNCHANGED:** The file is still identical.
- **MODIFIED:** The file exists but its content has changed.
- **NEW:** The file was not present when the baseline was created.
- **DELETED:** The file existed in the original baseline but is no longer present.

This allows users to quickly identify changes in a monitored folder.

3. Features of the Tool

3.1 Baseline Creation

FileGuard creates a trusted baseline by scanning all files in the selected folder and storing their SHA-256 hash values.

3.2 Recursive Folder Scanning

The tool uses recursive directory traversal to scan files inside the selected folder and all of its subfolders.

3.3 SHA-256 Hashing

The tool uses SHA-256 cryptographic hashing to create a unique fingerprint for each file.

Even a small change in a file generally produces a completely different hash value.

3.4 File Modification Detection

If the current hash of a file is different from the hash stored in the baseline, FileGuard identifies the file as modified.

3.5 New File Detection

If a file exists during the current scan but does not exist in the original baseline, FileGuard identifies it as a new file.

3.6 Deleted File Detection

If a file exists in the baseline but cannot be found during the current scan, FileGuard identifies it as deleted.

3.7 Support for Different File Types

Because files are read in binary mode, the tool can calculate hashes for many different file types, including:

- TXT
- JPG
- PNG
- PDF
- DOCX
- ZIP
- EXE
- MP4
- Configuration files
- Other binary and non-binary files

The tool does not need to understand the internal content of a file. It only calculates the cryptographic hash of its contents.

3.8 User Confirmation

When a new file is detected, the user can decide whether the file is legitimate and should be added to the baseline.

When a deleted file is detected, the user can confirm whether the deletion was intentional.

This prevents the tool from automatically trusting every change.

3.9 Severity Classification

The tool classifies the scan result according to the detected changes.

For example:

- **LOW:** No changes detected.
- **MEDIUM:** New or deleted files detected.
- **HIGH:** Existing files have been modified.
- **CRITICAL:** Significant combinations of modifications and deletions are detected.

3.10 Security Report

The tool generates a visual report containing:

- Total files scanned
- Number of unchanged files
- Number of modified files
- Number of new files
- Number of deleted files
- Overall security severity
- Detailed findings

4. Programming Language and Technologies Used

4.1 Python

Python was selected as the primary programming language because it provides simple and powerful libraries for file handling, cryptography-related operations, and data processing.

4.2 hashlib

The hashlib module is used to calculate SHA-256 hashes.

Example:

```
file_hash = hashlib.sha256(content).hexdigest()
```

4.3 os

The os module is used for:

- Accessing folders
- Creating file paths
- Scanning directories
- Recursively visiting subfolders

The os.walk() function is particularly important for scanning nested directories.

4.4 json

The json module is used to save and load the baseline file.

The baseline contains file paths and their corresponding SHA-256 hashes.

4.5 HTML and CSS

HTML and CSS are used to generate a visual security report that can be opened in a web browser.

5. System Workflow

The overall workflow of FileGuard is:

User selects a folder



Tool scans the folder recursively



SHA-256 hash is calculated for every file



Current hashes are compared with baseline



Changes are classified



New/deleted file confirmation is requested



Severity is calculated



Security report is generated

The tool has two main operating stages.

Stage 1: Baseline Creation

During the first scan, FileGuard:

1. Receives a folder path from the user.
2. Scans all files and subfolders.
3. Calculates the SHA-256 hash of each file.
4. Stores the relative file path and hash.
5. Saves the information in the baseline file.

Example:

```
{  
  "report.pdf": "SHA256_HASH_VALUE",  
  "images/photo.jpg": "SHA256_HASH_VALUE",  
  "documents/report.docx": "SHA256_HASH_VALUE"  
}
```

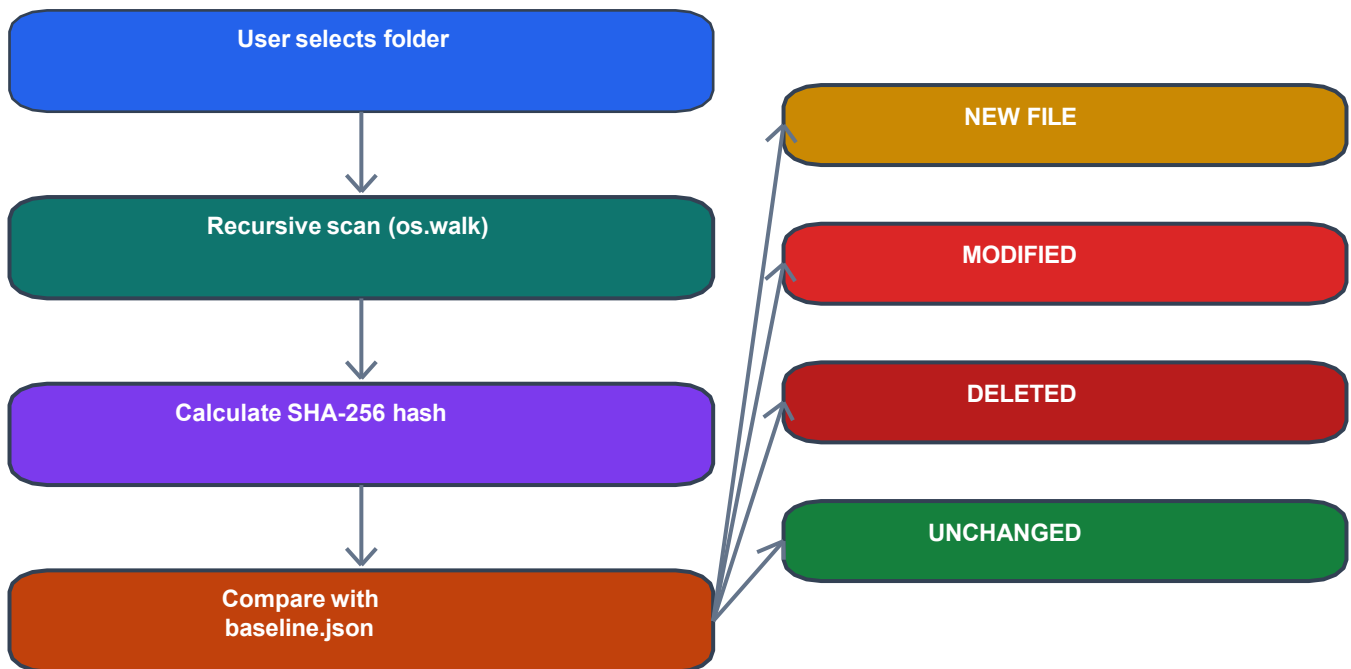
Stage 2: Integrity Scan

During a later scan:

1. The same folder is scanned again.
2. Current hashes are calculated.
3. Current hashes are compared with the baseline.
4. Differences are identified.
5. The files are classified.
6. A report is generated.

6. System Flow Diagram

The basic system flow is:



7. Implementation Details

7.1 File Hash Calculation

We have a function names as `calculate_hash` by which the tool opens each file in binary mode:

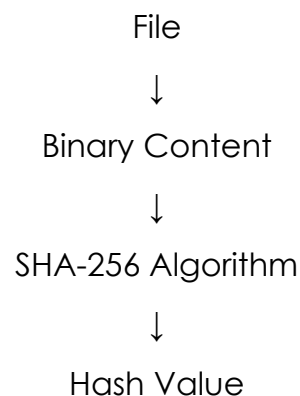
```
def calculate_hash(file_path):  
    file = open(file_path, "rb")  
  
    content = file.read()  
  
    file.close()  
  
    file_hash = hashlib.sha256(content).hexdigest()  
  
    return file_hash
```

The "rb" mode means that the file is opened for reading as binary data.

This is important because FileGuard needs to support different types of files, including images, PDFs, videos, and executable files.

After reading the content, the SHA-256 hash is calculated and then the resulting hash is returned.

The process can be summarized as:



7.2 Recursive Folder Scanning

FileGuard uses `os.walk()` to scan the selected folder and all subfolders.

```
for root, directories, files in os.walk(folder):  
  
    for file in files:  
  
        file_path = os.path.join(root, file)  
  
        relative_path = os.path.relpath(file_path, folder)  
  
        file_hash = calculate_hash(file_path)  
  
        baseline[relative_path] = file_hash
```

The `os.walk()` function provides:

- The current folder path
- The directories inside the current folder
- The files inside the current folder

This allows FileGuard to scan a structure such as:

```
ImportantFolder/  
|  
├─ report.pdf  
├─ image.jpg  
|  
├─ Documents/  
|   └─ file1.docx  
|       └─ file2.pdf  
|  
└─ Backup/  
    └─ config.txt
```

The tool can scan all files in the structure.

7.3 Relative File Paths

Instead of storing only the filename, FileGuard stores the relative path.

For example:

Documents/report.pdf

This is better than storing only:

report.pdf

because two different subfolders may contain files with the same name.

Example:

Folder1/report.pdf

Folder2/report.pdf

These are two different files, so their relative paths allow FileGuard to distinguish them.

7.4 Baseline Storage

The baseline is stored in JSON format.

Example:

```
{  
  "report.pdf": "hash_value_1",  
  "Documents/report.pdf": "hash_value_2",  
  "images/photo.jpg": "hash_value_3"  
}
```

The baseline represents the trusted state of the folder at a specific point in time.

8. Step-by-Step Explanation of the Tool

Step 1: User Selects an Operation

The user selects whether to:

- Create a new baseline
- Scan an existing baseline

Step 2: User Provides a Folder Path

The user enters the folder path that should be monitored.

Example:

```
=====
FILEGUARD
File Integrity Monitoring Tool
=====

1. Create Baseline
2. Scan Folder

Select an option (1/2): 2

Enter the complete path of the folder to monitor:
E:\Git Codings\File Guard
```

The tool can also scan folders containing nested directories.

Step 3: Baseline Creation

The tool scans every file and calculates its SHA-256 hash.

The results are stored in the baseline.

```
file_guard.py  {} baseline.json X  <>
E: > Git Codings > File Guard > {} baseline.json > ...

1  {
2    "files\\test1.txt": "158a0b85569f9cf3a3af3d581bc212235488bab575653a7751f87c083d900597",
3    "files\\test2.txt": "158a0b85569f9cf3a3af3d581bc212235488bab575653a7751f87c083d900597",
4    "files\\test3.txt": "5004334b3130ef39d4f01c63446bdc9f93c947ab238c11522f000b715b886126",
5    "files\\test4.txt": "158a0b85569f9cf3a3af3d581bc212235488bab575653a7751f87c083d900597"
6  }
```

Step 4: A File Is Changed

For testing purposes, a file can be modified after creating the baseline.

For example:

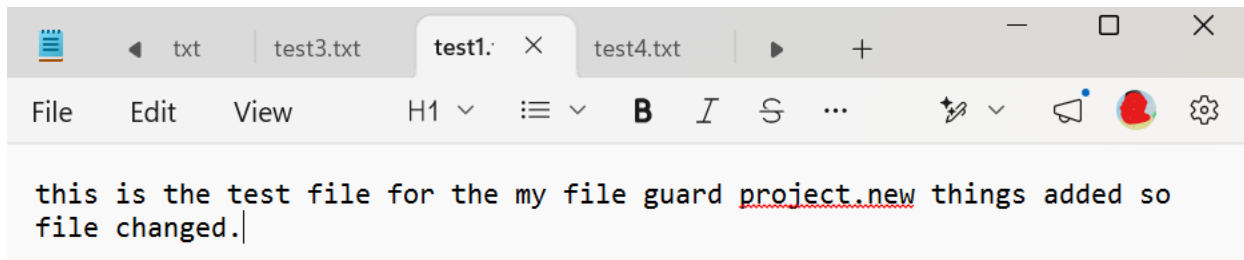
Test1.txt

may initially contain:

```
File  Edit  View  H1  ≡  B  I  S  ...  ↗  🔊  🍌  ⚙️

this is the test file for the my file guard project.
```

And after modification:



so its SHA-256 hash will also change.

```
FILEGUARD SECURITY SCAN
=====
[+] Scanning folder: E:\Git Codings\File Guard
[+] Scan started...

[MODIFIED]   baseline.json
[MODIFIED]   fileguard_report.html
[UNCHANGED]  file_guard.py
[MODIFIED]   files\test1.txt
[UNCHANGED]  files\test2.txt
[UNCHANGED]  files\test3.txt
[UNCHANGED]  files\test4.txt

=====
SCAN SUMMARY
=====
Files checked: 7
Unchanged:     4
Modified:      3
New:           0
Deleted:       0
Severity:      HIGH
=====
[+] HTML report saved as: fileguard_report.html
```

Ignore the other modified files like baseline and report as they always will be changed when hash of any file is changed.

Step 5: A New Scan Is Performed

The tool scans the folder again and calculates the current hash of every file.

Step 6: Hashes Are Compared

The current hash is compared with the stored hash.

```
current_hash = calculate_hash(file_path)
if relative_path in baseline:
    old_hash = baseline[relative_path]
    if current_hash == old_hash:
        print("[UNCHANGED] ", relative_path)
        unchanged_count += 1
    else:
        print("[MODIFIED] ", relative_path)
        modified_count += 1
        findings.append("[MODIFIED] " + relative_path)
```

Step 7: New and Deleted Files Are Detected

If a current file does not exist in the baseline then there will be NEW and if a baseline file does not exist during the current scan then DELETED.

```
[UNCHANGED] file_guard.py
[MODIFIED]   files\test1.txt
[UNCHANGED] files\test2.txt
[NEW]        files\test3 - Copy.txt
Do you want to save the hash of this new file? (y/n): y
[+] File added to baseline.
[UNCHANGED] files\test3.txt

[DELETED]    files\test4.txt
Did you intentionally delete this file? (y/n): y
[+] File removed from baseline.
```

Step 8: User Confirmation

The user is asked whether:

- A newly detected file should be added to the trusted baseline.
- A deleted file was intentionally deleted and should be removed from the baseline.

This gives the user control over the baseline.

```
Do you want to save the hash of this new file? (y/n): y
[+] File added to baseline.

[DELETED]    test1.txt
Did you intentionally delete this file? (y/n): y
[+] File removed from baseline.

[DELETED]    test2.txt
Did you intentionally delete this file? (y/n): y
[+] File removed from baseline.

[DELETED]    test3.txt
Did you intentionally delete this file? (y/n): y
[+] File removed from baseline.

[DELETED]    test4.txt
Did you intentionally delete this file? (y/n): y
[+] File removed from baseline.
```

Step 9: Report Generation

After the scan, FileGuard generates a security report containing the results.

FileGuard Security Report

File Integrity Monitoring Tool

Scan Time: 2026-07-23 16:17:47.218546

Folder: E:\Git Codings\File Guard

7

Files Checked

3

Unchanged

3

Modified

1

New

1

Deleted

Security Status: CRITICAL

Security Findings

[MODIFIED] baseline.json

[MODIFIED] fileguard_report.html

[MODIFIED] files\test1.txt

[NEW] files\test3 - Copy.txt

[DELETED] files\test4.txt

9. Detection Logic

The detection logic is:

Condition	Result
File exists in baseline and hashes are equal	UNCHANGED
File exists in baseline but hashes are different	MODIFIED
File exists currently but not in baseline	NEW
File exists in baseline but not currently	DELETED

10. Severity Classification

FileGuard uses the scan results to assign an overall severity level.

LOW

No changes are detected.

Modified = 0

New = 0

Deleted = 0

MEDIUM

New or deleted files are detected, but no existing files have been modified.

HIGH

One or more existing files have been modified.

Modification of an existing file may be more significant because the file was already trusted in the original baseline.

CRITICAL

A combination of serious changes is detected, such as:

- Modified files
- Deleted files
- Multiple suspicious changes

The severity level helps the user quickly understand the importance of the scan result.

```
210     if modified_count > 0 and deleted_count > 0:
211         severity = "CRITICAL"
212
213
214     elif modified_count > 0:
215         severity = "HIGH"
216
217
218     elif new_count > 0 or deleted_count > 0:
219         severity = "MEDIUM"
220
221
222     else:
223         severity = "LOW"
224
```

11. Challenges Faced and Their Solutions

Challenge 1: Understanding File Hashing

Initially, understanding how a file could be represented by a hash was a challenge.

This was resolved by studying cryptographic hashing and understanding that a hash is generated from the file's contents.

The SHA-256 algorithm was selected to generate file fingerprints.

Challenge 2: Supporting Different File Types

A challenge was ensuring that the tool could scan files such as images, PDFs, and other binary formats.

This was resolved by opening files in binary mode using:

```
open(file_path, "rb")
```

This allows the hashing process to work with different file types.

Challenge 3: Scanning Nested Folders

A simple folder listing only scans files in one directory.

This was improved by using:

```
os.walk()
```

This allows the tool to recursively scan all subfolders.

Challenge 4: Duplicate File Names

Two different folders may contain files with the same name.

For example:

```
Folder1/report.pdf
```

```
Folder2/report.pdf
```

This problem was resolved by storing relative paths instead of filenames alone.

Challenge 5: Handling New Files

A new file is not automatically trusted.

The user is asked whether the new file should be added to the baseline.

This gives the user control over the trusted state.

Challenge 6: Handling Deleted Files

A deleted file may have been intentionally removed by the user.

Therefore, the tool asks the user whether the deletion was intentional before updating the baseline.

Challenge 7: Creating Professional Output

Basic terminal output is useful but not always easy to understand.

To improve usability, FileGuard includes:

- Counters
- Findings
- Severity classification
- A visual security report

12. Limitations

The current version of FileGuard has some limitations.

12.1 Baseline Protection

If an attacker can modify both the monitored files and the baseline file, the integrity check could potentially be bypassed.

A future version could digitally sign the baseline or store it in a protected location.

12.2 Manual Scanning

The current tool requires the user to run the scan manually.

Future versions could support scheduled scans.

12.3 No Real-Time Monitoring

The current version detects changes when a scan is performed.

It does not continuously monitor the file system in real time.

12.4 Local Operation

The current version is designed for local file integrity monitoring.

Future versions could support centralized monitoring of multiple systems.

12.5 Baseline Path Consistency

FileGuard currently relies on the folder path provided by the user during scanning. If a baseline is created using a parent path, such as:

Documents/Test

but the user later scans the same folder using a different path representation, such as:

Test

the tool may treat the files as different because their stored relative paths do not match the paths generated during the new scan.

This limitation occurs because the current version does not automatically normalize different path representations or permanently associate a baseline with its original folder location. Therefore, the user should provide the same folder path structure used when creating the baseline.

A future version could improve this by storing the absolute path of the monitored folder in the baseline and automatically validating or normalizing the path before scanning.

14. Future Improvements

Possible future improvements include:

- Digital signatures for the baseline
- Real-time file monitoring
- Scheduled automatic scans
- Email alerts
- Web dashboard
- User authentication
- Multiple monitored folders
- Configurable exclusions
- Database-based baseline storage
- Centralized monitoring
- Integration with SIEM systems

14. Conclusion

FileGuard demonstrates the practical application of cybersecurity concepts through a functional software tool.

The project applies:

- Cryptographic hashing
- SHA-256
- File system traversal
- Recursive directory scanning
- Baseline creation
- File integrity monitoring
- Change detection
- User confirmation
- Severity classification
- Security report generation

The tool can identify whether files have remained unchanged, been modified, newly created, or deleted.

Although the implementation is intentionally simple and understandable, it demonstrates an important cybersecurity concept used in real-world security systems: file integrity monitoring.

The project also demonstrates how Python can be used to develop a practical cybersecurity tool using standard libraries and relatively simple programming logic.

16. GitHub Repository

<https://github.com/mrzeeshi/File-Guard>

The repository contains:

- Complete Python source code
- Baseline functionality
- File scanning functionality
- Hash calculation logic
- Report generation code
- README documentation
- Instructions for running the tool